application.js has the part of the setup that is not auto-generated.

Let's look at controllers/application.js for a second:

chapter_03/01/app/javascript/controllers/application.js

```
import { Application } from "@hotwired/stimulus"

const application = Application.start()

// Configure Stimulus development experience
application.debug = false
window.Stimulus = application

export { application }
```

It imports an Application object and starts it up. It then sets a debug attribute to false. If the debug attribute is true, then Stimulus will be very chatty in the console log, telling you every time Stimulus logs a controller connection or action. (By the time you read this, there may also be a warnings attribute, but it has, at least temporarily, been removed.) Finally, Stimulus sets up global access to the application object at window.Stimulus. Among a couple of other things, this allows you to turn the debugger on or off globally. If you change the debug value, you need to restart the server for the change to take effect.

Back to the rest of the index.js file. Exactly what the installer puts after loading the application depends on how we are bundling the Stimulus controllers. The basic idea is to import all the controllers in the directory so that they are available to the browser and to the Stimulus engine.

If we were using webpack, this would be easy, because webpack allows you to group import using file glob syntax. The only change we'd need to make to the version of this file that would be installed if we were using webpack is that we'd have to change the file extension to include .ts as a possibility.

However, we're not using webpack, we're using esbuild, and esbuild does not allow imports using glob syntax. Instead, Stimulus generates a manifest of existing controllers which you can trigger at any time using the command